

自适应自动框架：面向开放式任务流中智能体系统部署的持续自我改进

Zewen Liu¹, Zhan Shi², Yisi Sang², Bing He², Minhua Lin³, Tianxin Wei⁴

Dakuo Wang⁵, Benoit Dumoulin², Wei Jin¹, Hanqing Lu²

¹Emory University ²Amazon ³The Pennsylvania State University ⁴UIUC ⁵Northeastern University
{zewen.liu,wei.jin}@emory.edu; luhanqin@amazon.com

摘要

诸如 A-Evolve、GEPA 和 Meta-Harness 等自动框架系统通过从执行反馈中优化提示、技能、工具、记忆及支撑基础设施来改进大语言模型智能体，但它们通常在固定的离线基准上进行评估。实际部署则呈现开放式任务流：历史记录不断增长且无固定终点，异构任务需要不同的框架，且问题分布随时间变化。这些挑战使得单一且频繁密集更新的框架变得脆弱，导致性能下降，表现为准确率早期达到峰值后逐渐衰退。这促使我们采用任务级自适应的持续框架构建方法。我们提出了自适应自动框架（Adaptive Auto-Harness），一个面向此类任务流的框架与系统。该框架将相对于理想框架的差距分解为演化损失和自适应损失。系统通过一个有状态的多智能体演化器、一个带求解时路由的框架树，以及在历史记录缺乏所需信号时的人工引导钩子来解决这些损失。在预测市场、安全竞赛和事件预测任务流中，自适应自动框架优于五个现有的自动框架基线，消融实验将收益归因于更好的构建、路由或针对性的人工引导。代码见链接。

1 引言

开放式任务流是大语言模型智能体常见的部署场景：任务持续到达，反馈随时间累积，且未来任务可能与早期任务不同。在此场景下，智能体的框架——包括围绕固定大语言模型的提示、技能、工具和支撑基础设施——是任务求解性能的主要决定因素。诸如 A-Evolve (Lin 等, 2026)、GEPA (Agrawal 等, 2025) 和 Meta-Harness (Lee 等, 2026) 等自动框架系统通过演化来自动构建框架

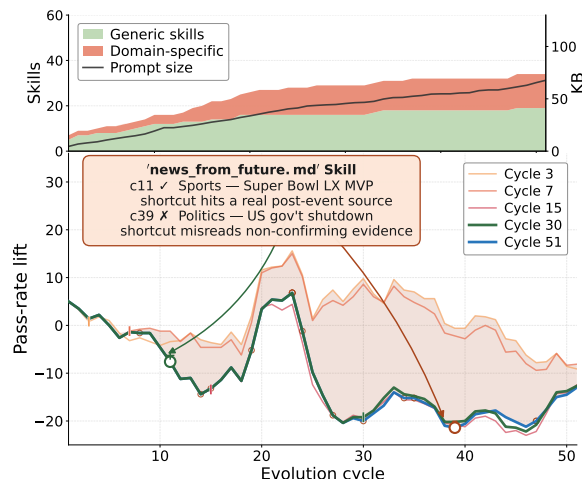


图 1：更长时间、更频繁的演化可能过度拟合早期任务流证据。上图：无界演化的 A-Evolve 运行中，技能从 12 个增长到 34 个，而提示从 2 KB 增长到 68 KB。下图：每条曲线报告相对于无演化求解器的通过率提升，采用不同的演化停止周期。早期收益随着后续任务的到来而消退；news_from_future.md 在体育任务上有帮助，但在政治任务上失效。

从执行反馈中学习，并在静态离线基准（如 SWEbench (Jimenez 等, 2024)）上报告了显著提升。然而，这些评估并未捕捉到部署的核心压力：系统必须在不断运行的时间流上持续改进，其历史不断增长、任务类型多样且分布发生偏移。代表性流包括数周内包含数千个问题的预测市场 (Cheng 等, 2026)、长达十年的安全竞赛 (Zhuo 等, 2025) 以及具有异构来源的跨语言预测服务 (Zeng 等, 2025)。图 1 说明了为何在求解过程中反复演化并注入所有系统状态是不充分的。在 Polymarket 上：我们运行 A-Evolve 并在 3、7、15、30 或 51 个周期后停止演化，将每次运行与未演化的相同求解器进行比较。早期演化提高了通过率，但更长的运行积累了更大的提示和更专业的技能，仅

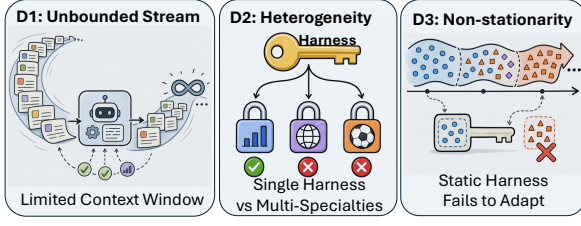


图 2：开放式任务流中的三个部署维度。无界流、异构任务和非平稳分布暴露了为长期部署演化单一密集系统状态的局限性

其中只有部分技能能够迁移。例如，一个有用的技能 `news_from_future.md`（138 次正确对 16 次错误的买入）在体育任务上有帮助，但在政治任务上却会出错。所有停止预算最终都会达到峰值然后下降；在流的后期，较短的运行优于较长的运行。因此，持续部署需要在保留有用历史的同时，使活跃系统状态适应当前任务。

这一失败暴露了静态基准评估无法捕捉的三个部署维度（图 2）。（D1）无界流。任务流没有固定的训练/测试划分或指定终点（Karten 等，2026；Wang 等，2023）；反馈、轨迹和系统状态在整个部署过程中不断累积，给演化带来了沉重负担。现有的围绕单智能体演化器构建的自动系统状态系统将这种不断扩展的历史压缩到有限的上下文窗口中，从而在构建有效且可泛化的系统状态时形成瓶颈。（D2）任务

异构性。不同类型的任务混合在同一流中。例如，预测市场平台在同一小时内混合了政治、体育和金融问题，每个问题都需要不同的来源、工具和提示。然而，现有的自动框架系统在整个流中部署静态密集框架，在求解时不对当前任务进行适应，并且忽视了一个事实：单一固定策略在异构问题上很少是最优的（Miao 等，2025）。

（D3）分布性

非平稳性。随着流的推进，传入的任务偏离了框架最近拟合的经验。因此，即使拥有丰富的历史经验，为最近周期优化的框架也会对新任务失去拟合。弥合这一差距需要对框架进行逐任务的上下文适应，而不仅仅是持续的历史拟合。更多诊断见附录 A。

我们通过一个统一的

分析框架来解决这些维度，并识别出先前自动框架系统忽视的两个根本差距。该分析促使：（1）持续自动框架（§ 3.3），用具有跨周期知识的有状态多智能体系统取代无状态的一次性进化，以更好地构建框架；（2）求解时适应（§ 3.4），在求解前为每个问题适应任务相关的框架，在异构、漂移的流上恢复逐任务拟合。除了这两个轴，我们进一步引入第三个轴：人在回路（HITL），用于辅助引导框架，纳入历史经验中缺失的人类洞见和预见。我们的贡献是：

（1）开放式智能体流的部署机制分析。我们形式化了为什么当任务以无界、异构和非平稳流的形式到达时，静态自动框架评估是不充分的。该框架将最优框架的差距分解为进化损失和适应损失，为自动框架系统设计提供指导。

（2）自适应自动测试框架系统。本文引入有状态的多智能体演化器以实现持续的测试框架构建，引入测试框架树路由器以实现求解时的自适应调整，并引入结构触发的人在回路钩子，以便在历史经验不足时进行演化。（3）全面的实证验证与诊断。我们在三个流式任务上进行了评估，这些任务涵盖预测市场、安全挑战和事件预测，并与其它自动框架系统进行了对比。除了聚合性能之外，我们还针对所提出的差距、组件消融以及人在回路切片分析提供了深入的分析 and 证据。

2 相关工作

任务流中的持续学习。持续学习研究从一系列任务中学习并保留早期能力的系统（Buzzega 等，2020；Wang 等，2022b,a）。领域自适应与测试时自适应解决分布偏移问题（Ben-David 等，2010；Ganin 等，2016；Wang 等，2020；Liang 等，2020），而专家混合方法将异构输入路由到专用组件（Jacobs 等，1991；Shazeer 等，2017）。这些方向覆盖了我们 D1 – D3 设定的部分内容，但它们通常调整模型权重、分类器或专家模块。相比之下，我们研究的是框架层面的类似机制。

自我改进与自我进化的智能体。与本文设定最接近的大语言模型智能体先例是直接从执行反馈中更新框架的系统。A-Evolve (Lin 等人, 2026) 引入了一种线性链演化器：每个循环读取批量轨迹，并对提示、技能、记忆和工具进行变异。GEPA (Agrawal 等人, 2025) 利用文本反馈而非标量奖励，增加了反思性的帕累托提示演化。Meta-Harness (Lee 等人, 2026) 使用不断增长的文件系统档案，并以 Claude Code 作为提议者。Continual Harness (Karten 等人, 2026) 通过交替的行动/细化循环，在单个连续部署运行中实现在线适应。SkillOS (Ouyang 等人, 2026) 通过强化学习学习技能管理策略，训练一个管理者从重复交互中选择并细化可复用技能。

3 方法

3.1 问题形式化

本文考虑以开放式流形式到达的任务 $x_1, x_2, \dots, x_T$ ，其中 $x_t \sim P_t$ ， P_t 为时刻 t 的任务分布，且每个 x_t 具有固定的真实标签 $y(x_t)$ 。智能体观察历史经验 $\mathcal{H}_t = \{(x_i, r_i, \tau_i)\}_{i=1}^{t-1}$ ，其中 r_i 可选地为任务 x_i 上已实现的奖励， τ_i 为智能体的求解轨迹（动作、中间观测、工具调用）。一个框架 $C = \varphi(\mathcal{H}_t)$ ，具有 $|C| \leq K$ ，是在容量预算 K 下的有界表示（提示、技能、记忆和工具），其中 φ 为进化器：自动将历史经验转化为框架的智能系统。随后，求解智能体通过策略 $a_t \sim \pi(a | x_t, C_t)$ 行动，其中 π 为以任务和框架为条件、由大语言模型诱导的动作采样分布。

3.2 分析框架

第 1 节中的三个部署维度揭示了具体失败模式，但并未指明进化器应修复什么。为定位根本原因，本文从分析角度形式化该问题：将框架构建视为针对预言机参考的遗憾最小化问题，并将遗憾分解为两个互补的损失项，直接映射到可操作的设计轴。

效用与遗憾。本文定义框架 C 在任务 x_t 上的效用为

$$V(C, x_t) = \mathbb{E}_{a_t \sim \pi(\cdot | x_t, C)}[r(a_t, y(x_t))], \quad (1)$$

即框架条件下求解器的期望奖励。全历史效用为容量预算下相应的上限，

$$V(\mathcal{H}_t, x_t) := \sup_{C: |C| \leq K} V(C, x_t), \quad \text{即}$$

任何有界驾驭在上可达到的最佳效用。演化器 x_t 构建的驾驭的遗憾则为

$$\text{Regret}(\varphi, x_t) = V(\mathcal{H}_t, x_t) - V(\varphi(\mathcal{H}_t), x_t) \geq 0, \quad (2)$$

非负，因为 $\varphi(\mathcal{H}_t)$ 是上确界域中的一个元素。从操作上讲，该上限与求解器在 φ 相同的计算和工具预算下直接获得 $\mathcal{H}_t$ ，并通过即时重建任何候选驾驭所能达到的水平一致。求解时最优驾驭。固定演化器类 Φ ，并针对特定任务定义求解时最优驾驭：

$$C_{\Phi}^*(x_t) = \arg \max_{\substack{C = \varphi(\mathcal{H}_t, x_t), \varphi \in \Phi \\ |C| \leq K}} V(C, x_t). \quad (3)$$

这是一个假设的预言参考：它是演化器类 Φ 在求解时被允许根据传入任务 x_t 进行条件化时所能产生的最佳驾驭。部署的演化器 φ 在观察到 x_t 之前就承诺了一个驾驭 $\varphi(\mathcal{H}_t)$ ；以 C_{Φ}^* 作为 $V(\mathcal{H}_t, x_t)$ 和 $V(\varphi(\mathcal{H}_t), x_t)$ 之间的枢轴，得到如下分解。

命题 1 (遗憾分解)。对于任何部署的演化器 $\varphi \in \Phi$,

$$\mathbb{E}_{x_t}[\text{Regret}(\varphi, x_t)] = L_{\text{evo}}(\Phi) + L_{\text{adapt}}(\varphi), \quad (4)$$

其中

$$L_{\text{evo}}(\Phi) = \mathbb{E}_{x_t}[V(\mathcal{H}_t, x_t) - V(C_{\Phi}^*(x_t), x_t)], \quad (5)$$

$$L_{\text{adapt}}(\varphi) = \mathbb{E}_{x_t}[V(C_{\Phi}^*(x_t), x_t) - V(\varphi(\mathcal{H}_t), x_t)]. \quad (6)$$

L_{evo} 是演化损失：它反映了演化器类 Φ 的能力差距，与演化器运行多少个周期无关。单代理提示编辑器无法生成多文件基础设施；该上限是结构性的，而非努力程度问题。因此，减少 L_{evo} 需要追求能力更强的演化器系统，并使其获得更广泛的控制和反馈。 L_{adapt} 是适应损失：预言机为每个任务构建最优驾驭，但部署的 φ 在观察到 x_t 之前就承诺了一个驾驭。因此，即使拥有最优的演化器系统，只要任务异质性持续存在， L_{adapt} 就会存在。

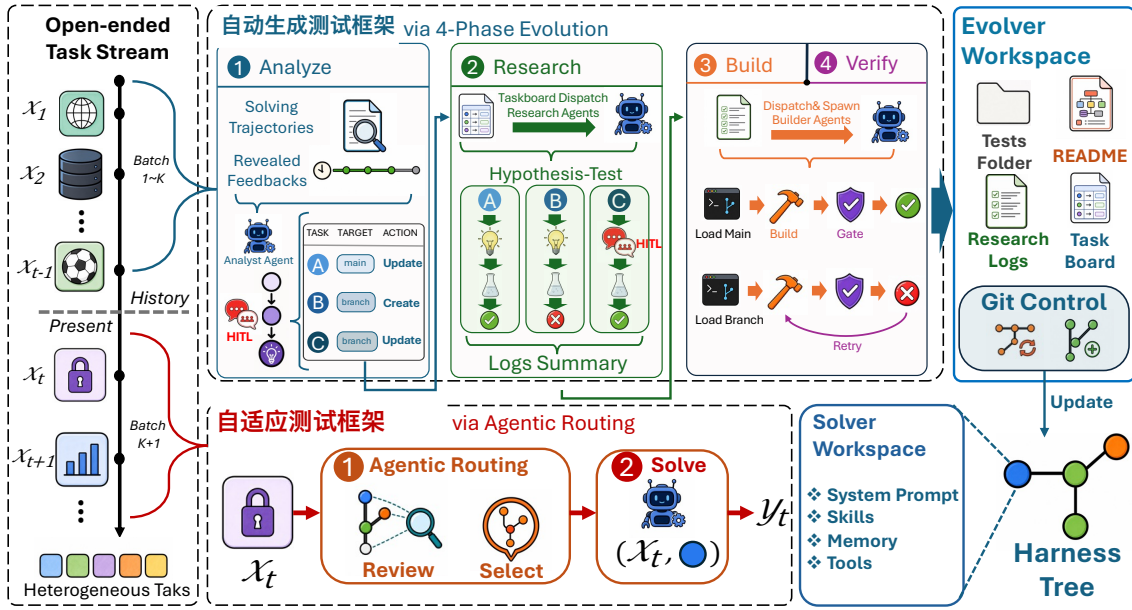


图 3：自适应自动测试框架系统概览。顶部：多智能体进化器通过四个阶段（分析 → 研究 → 构建 → 验证）在多个周期内构建并优化测试框架树，并配备持久的跨周期工作区和时序揭示反馈。底部：在求解时，路由智能体通过 git show 读取每个分支的工作区，并将传入任务 x_t 路由到最合适的分支。两个人在回路钩子（任务板引导和研究阶段辅助）仅在进化器的历史缺乏相关信号时触发。

人在回路作为第三轴。该分解假设 $\mathcal{H}_t$ 包含与相关的信号。当不包含时，遗憾分解不再适用； x_t 我们通过 $L_{\text{evo}} + L_{\text{adapt}}$ 之外的第三轴，即人在回路通道（§ 3.5）来处理这种情况。

3.3 通过多智能体实现持续自动测试框架进化

分析分解将 L_{evo} 识别为进化器类无法从历史中构建的测试框架能力所造成的损失。我们通过扩展 Φ ，引入有状态的四阶段多智能体进化器、时序揭示反馈和跨周期记忆来减少这一损失（图 3）。该设计针对无界流失败，即单智能体进化器必须在一个上下文窗口内吸收不断增长的轨迹、延迟标签和先前研究。具体而言，我们解决三个结构性限制：（1）具有不同角色和目标的多智能体。由于无界流不断扩展进化器必须解释的轨迹历史，现有的单智能体进化器必须将分析、研究、实现和验证全部塞入一个上下文窗口。我们将进化分解为四个阶段（分析 → 研究 → 构建 → 验证），每个阶段都有专门的目标和完整的上下文预算。

这消除了单窗口瓶颈，并允许并行的研究者探索互不相交的假设，而不会过早收敛。

（2）时间揭示反馈。在无界流下，标签异步到达（例如，预测市场在交易后数天才结算）。我们实现了一个时间揭示门，仅在每个任务的解决日期之后才将其评估信号呈现给进化器，从而提供适当的流式反馈信号，而不会泄露未来信息。

（3）跨周期持久状态。我们为进化器提供了一个跨周期持续存在的专用工作区，其中包含任务板（优先失败分析）、研究日志（带有通过/失败结论的已测试假设）、架构文档（README）和验证测试。这种跨周期记忆使进化器能够随时间改进其构建能力，并建立在先前进化经验的基础上，而不是从头开始。

3.4 通过挂载树进行求解时自适应

路由分析分解将 L_{adapt} 识别为在观察到传入任务上下文之前就承诺使用一个密集挂载器所造成的损失。我们通过将繁重的自适应转移到进化时间来减少这种损失：进化器构建一个结构

化的工具库存储，求解时间仅需轻量级适配算子。该约简具有一般性：它分解为 (i) 进化器如何组织工具库空间，以及 (ii) 求解器如何针对每个任务从该空间中进行选择。工具库空间可以多种形式组织，包括始终使用最近进化工作区的线性链、特定领域分支的树，或具有依赖边的技能图。适配算子同样可以采取多种形式，从扁平目录上的技能级检索（字节跳动，2025）到结构化空间上的分支级路由。不同组合在构建成本、算子延迟和适配发生的粒度之间进行权衡。我们采用工具库树作为存储形式，并采用智能体路由作为适配算子。树结构天然适合我们的场景，因为异构任务流会聚成少量重复出现的领域（例如，CTF-Dojo 中的二进制利用与密码学，PolyBench 中的体育与政治），分支隔离了特定领域的提示/技能/工具，避免相互污染，而分支门控为进化器提供了一个明确的杠杆，仅在失败证据证明有必要时才进行专门化。我们用两种设计实例化这一方案（图 3）。(1) 分支工具库树（在

进化时构建）。求解器的工作区是一个 git 仓库。进化器在进化过程中构建特定领域的分支（例如，branch/crypto-classical、branch/binary-reversing），每个分支携带自己的提示、技能和工具注册表；git 提供跨分支的版本控制、隔离和谱系跟踪。(2) 智能体**路由（在求解时执行）**。路由智能体通过 git show 读取每个分支的工作区，并根据 x_t 的上下文选择分支；然后求解器检出该分支并执行。

3.5 人在回路通道

第三类失败位于 $L_{\text{evo}} + L_{\text{adapt}}$ 之外：某些任务需要 $\mathcal{H}_t$ 信号中缺失的工具库，例如 API 凭据、新型网络来源或专有端点。在这种经验不足的情况下，无论是更强的进化器还是求解时的路由都无法恢复缺失的信号。我们通过一个人在回路通道来解决这一问题，该通道通过结构化触发的引导钩子来增强 $\mathcal{H}_t$ （图 3）。该设计针对开放式任务流，其中新的访问需求在自主进化获得相关证据之前就已出现。

表 1：三个按时间顺序排列的任务流的基准统计信息。

Bench.	Tasks	Span	Domain
PolyBench	5,075	Feb 6–22, 2026	Prediction markets
CTF-Dojo	261	2011–2024	Security
FutureX	503	Jan–Apr 2026	Forecasting

(1) 任务板引导。分析智能体更新任务板后，人类可审阅该板以添加条目、调整优先级或提供领域指导与数据源访问权限。此举主动引导后续研究周期，提供演化器仅凭轨迹无法推导出的方向。**(2) 研究过程中的交互式协助。**当研究智能体在执行中途遇到需要人工干预的障碍（例如身份验证墙）时，钩子会实时提示人类。此举在故障点处被动地解除研究智能体的阻塞。

4 实验

4.1 实验设置

基准。我们在三个开放式任务流上进行评估（表 1）：用于预测市场的 PolyBench（Cheng 等，2026）、用于安全挑战的 CTF-Dojo（Zhuo 等，2025）以及用于事件预测的 FutureX（Zeng 等，2025）。三者均强制执行严格的时间顺序，并覆盖挑战的三个维度。细节与非平稳性诊断见附录 A 和 C。

基线。我们与使用 **Sonnet-4.6**（Anthropic，2025b）、**DeepSeek-V3.2**（Liu 等，2025）的不同求解智能体的无演化运行进行比较，

Claude Haiku-4.（Anthropic，2025a）、**GLM-4.7**（Z.ai，2025）以及 **Kimi-K2.5**（Team 等，2026）。

本文与五种自动框架基线进行比较：**A-Evolve**（Lin 等，2026）、**GEPA**（Agrawal 等，2025）、**Meta-Harness**（Lee 等，2026）、**Continual Harness**（Karten 等，2026）和 **SkillOS**（Ouyang 等，2026）。此外，还与一种人工设计系统 **OctoTools**（Lu 等，2025）进行比较。

求解器与进化器。除非特别说明，所有实验均使用 Claude Sonnet 4.6 作为求解器，并使用 Claude Opus 4.6 作为进化器，两者温度均设为 0，以将性能提升归因于进化算法而非采样噪声；无进化对照组还报告了 Haiku 4.5、DeepSeek-V3.2、Kimi-K2.5 和 GLM-4.7 的基础智能体结果。所有算法共享相同的批大小（PolyBench/CTF-

表 2: 无进化智能体、自动框架基线、人工设计系统以及本文三种变体的主要比较。PolyBench 报告准确率/回报（覆盖率缩放的 CWR，以% 计）；CTF-Dojo 和 FutureX 报告官方 Pass@1。粗体标记每行最佳结果，下划线标记次佳结果。

Benchmark	Metric	No evolution					Auto-harness baselines					Human	Ours	
		Sonnet	Haiku	DeepSeek	Kimi	GLM	A-Evolve	GEPA	Meta Harness	Cont. Harness	SkillIOS	OctoTools	Multi agent	Full System
PolyBench	Accuracy $\uparrow$	22.2	15.2	1.4	14.0	14.0	18.4	13.4	50.8	8.5	21.4	40.0	<u>79.8</u>	80.9
	Return $\uparrow$	+1.7	+88.0	+16.4	-2.0	+10.1	+7.2	+0.2	+320	+1.7	+3.6	+20.4	<u>+351</u>	+330
CTF-Dojo	Pass $\uparrow$	37.2	23.8	26.1	24.5	12.6	45.2	42.9	41.0	25.7	29.5	38.3	<u>47.9</u>	50.2
FutureX	Pass $\uparrow$	31.0	31.0	31.2	27.8	30.8	<u>47.5</u>	28.2	29.4	31.8	29.8	25.6	49.5	47.3

Dojo/FutureX）、批循环和时间揭示门；仅进化算法不同。

度量指标。对于 PolyBench，本文报告两项互补度量：准确率（Accuracy），即所有市场中正确交易的比例；以及收益（Return），定义为覆盖率 $\times$ 置信加权收益（CWR），其中 CWR 为已交易市场的置信加权利润与投资之比，覆盖率为已交易市场的比例。对于 CTF-Dojo 和 FutureX，本文报告原始基准定义的官方通过率（Pass@1）。图中还报告了与基线的差值作为提升幅度。完整定义见附录 A。研究问题。**RQ1**（§ 4.2）：自适应自动框架（Adaptive Auto-Harness）在开放式任务流上与现有自动框架系统相比表现如何？**RQ2**（§ 4.3）：自适应自动框架如何解决基准特定的瓶颈？**RQ3**（§ 4.4）：带评估反馈的有状态多智能体进化能否带来额外增益？**RQ4**（§ 4.5）：求解时路由能否有效利用专用框架分支？**RQ5**（§ 4.6）：在历史信号不足时，人类引导能否提供帮助？

4.2 与基线的比较

现有系统各自专注于某一度量集群。A-Evolve 在两个通过率流（45.2% CTF-Dojo，47.5% FutureX）上领先，但仅覆盖 21.1% 的 PolyBench 市场。Meta-Harness 在 PolyBench 全部三项度量（55.3% 覆盖率，50.8% 准确率，+320% 收益）上领先，但在 FutureX 上低于无进化的 Sonnet 基线（29.4% vs. 31.0%）。基础求解器在每个模型上的 PolyBench 覆盖率均低于 32.6%，而 OctoTools 这一冻结的人工设计系统在 PolyBench 收益上排名第三，但未在任何一行领先。因此，通过率集群与投资组合集群位于不同的基线中。本文的三个变体共同在所有度量上领先。在无 HITL 干预的情况下，完整系统将多智能体进化与求解时路由相结合，达到 97.9% 的 PolyBench 覆盖率、80.9% 的准确率，

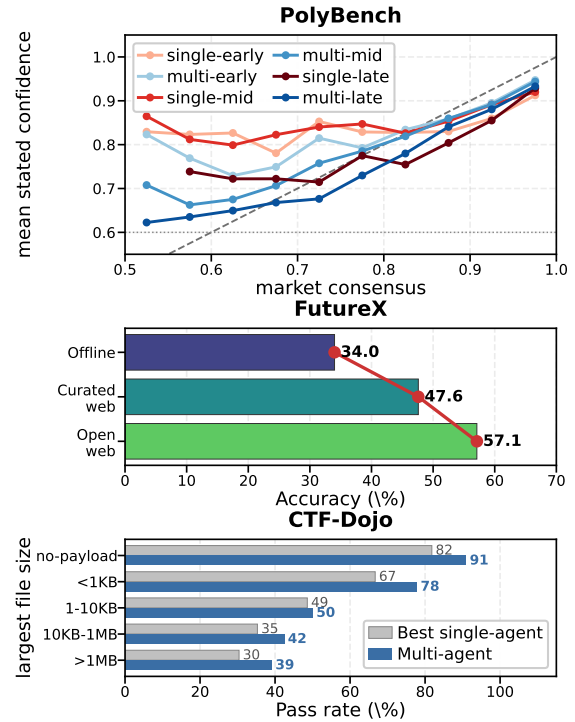


图 4: $\overline{L}_{\text{evo}}$ 基准特定瓶颈的证据。PolyBench 强调置信度校准，FutureX 强调网络检索访问，CTF-Dojo 强调不同文件大小的载荷处理。

以及 50.2% 的 CTF-Dojo 通过率。多智能体变体在 FutureX 上以 49.5% 领先，此时进化正确的来源和工具比逐任务路由更重要。自适应变体在 PolyBench 收益上以 +352% 领先，此时将每个市场匹配到专用策略更为关键。

4.3 基准瓶颈

研究问题二探讨自适应自动测试框架是否针对限制各基准的瓶颈。本文围绕第 3.2 节引入的两类损失展开分析：演化损失 L_{evo} ，即演化器类无法构建的能力差距；以及适应损失 L_{adapt} ，即在异构任务中固定使用单一测试框架所产生的差距。演化瓶颈（ L_{evo} ）在不同基准间存在差异

标记。图 4 绘制了特定基准的压力

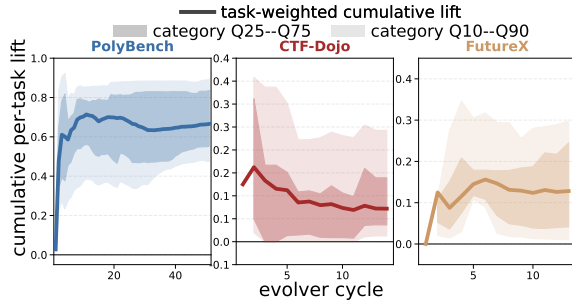


图 5: L_{adapt} 跨任务类别的证据。曲线展示了相对于基线的累积自适应提升, 阴影带表示各任务类别在周期内的性能分布。

轴选择以暴露最具预测能力。PolyBench 面板将平均陈述置信度与市场共识进行对比, 市场共识定义为 Polymarket 价格中偏好结果的隐含概率; 我们多智能体变体的良好校准线沿对角线分布, 而单智能体变体保持平坦并在低共识市场上过度陈述置信度, 表明关键能力是共识感知的置信度校准而非原始预测技能。FutureX 面板将通过率与三个检索层级进行对比, 范围从离线到日期过滤的维基百科加 DuckDuckGo 再到无限制的 DuckDuckGo 网络搜索。通过率从 34.0% 到 47.6% 再到 57.1% 单调递增, 表明关键能力是来源获取而非推理。CTF-Dojo 面板将通过率与最大挑战文件大小进行对比, 分为从无载荷到 > 1MB 的五个层级。最佳单智能体变体从 81.8% 下降到 30.4%, 多智能体变体从 90.9% 下降到 39.1%, 同时始终保持约 9 个百分点的优势, 表明随着输入增大, 载荷处理基础设施成为关键能力, 且多智能体进化器缓解但未消除这一瓶颈。总之, 关键能力因基准而异。适应瓶颈 (L_{adapt}) 仍然存在

进化。图 5 报告了各任务在进化周期中的自适应提升情况, 其中每个任务的提升为自适应框架得分减去单一固定框架基线得分的中位数。实线表示周期内平均提升, 阴影带表示跨任务类别的性能波动; 若单一固定密集框架足够, 则随着进化推进, 平均提升应趋近于零。然而我们观察到, 平均提升与波动在整个过程中始终保持为正

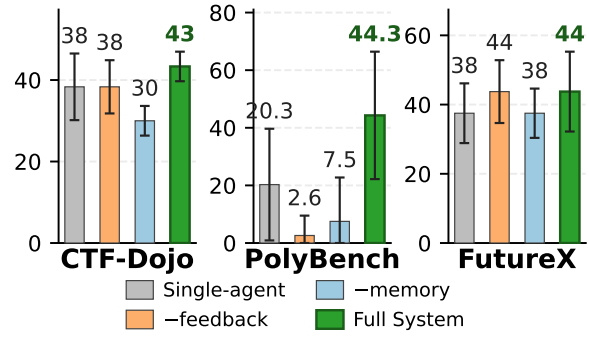


图 6: 多智能体演进的消融实验。与完整的有状态系统相比, 移除评估反馈或跨周期记忆会降低性能。

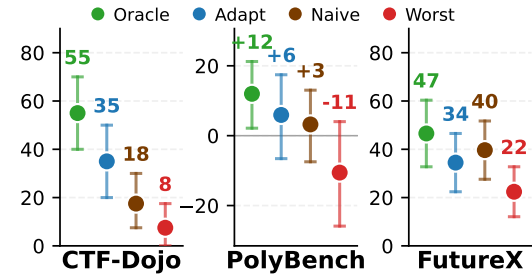


图 7: 在演进后的框架树上对求解时间路由进行的设计分析。我们为每个任务类别播种一个分支, 在数据流上演进树, 并将每个任务通过每个分支重放。Oracle 是每个任务的最佳分支, Adapt 是基于类别的路由策略, Naive 仅使用主工作区, Worst 是每个任务的最差分支。

在三个基准上的所有周期。

要点。尽管数据流暴露了不同的瓶颈, 但它们可以在同一原则下得到解决: 减少缺失能力的演进损失和任务特定适配的适应损失。

4.4 有状态多智能体演进

RQ3 询问第 § 3.3 节中的四角色演进器 (分析师 → 并行研究员 → 构建者 → 验证器) 是否优于单智能体演进器, 以及其状态通道是否重要。在 PolyBench/CTF-Dojo/FutureX 上使用

100/60/80 个任务, 图 6 将完整系统与移除时间揭示反馈或跨周期记忆的变体进行了比较。完整系统在三个基准上均优于单智能体演进器: 在 PolyBench、CTF-Dojo 以及 FutureX 上的 20.3 → 44.3 38% → 44% 中评估了 CWR。移除记忆导

致最广泛的性能下降, 而移除反馈主要影响 PolyBench, 因为该环境中结果在交易后才会确定。

要点。四角色进化器在与持久状态结合时表现最强: 记忆保留了

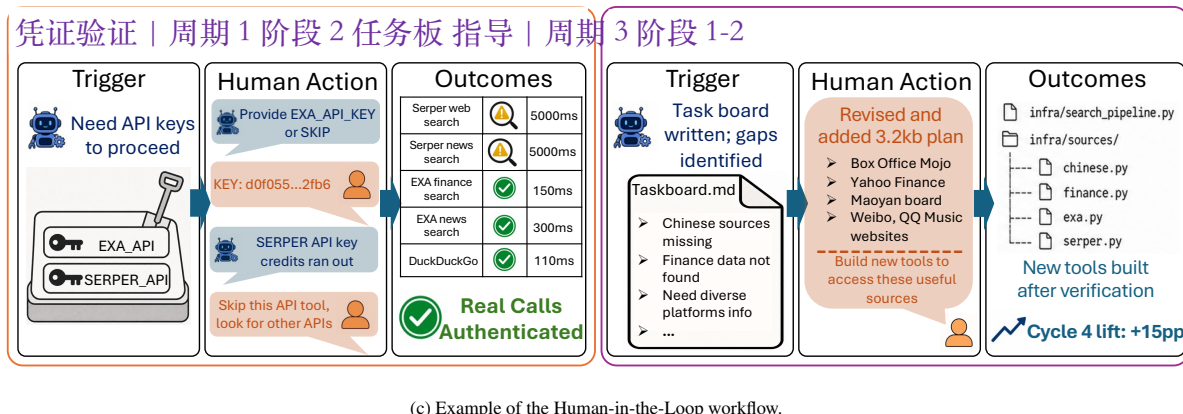
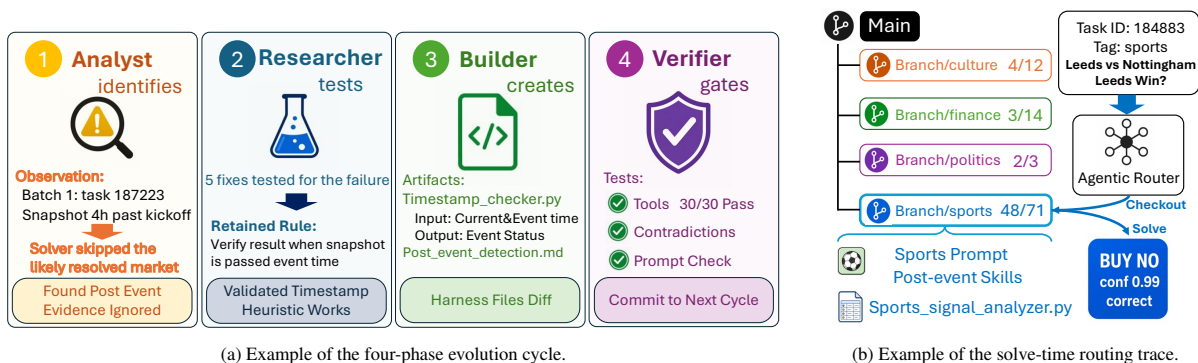


图 8：所设计的多智能体进化、智能体路由以及人在回路（Human-in-the-Loop）的提取轨迹。

跨周期搜索，而反馈将已解决的结果转化为后续进化的信号。

4.5 驾驭树上的求解时间路由

RQ4 探究专用驾驭树能暴露多少适应余量，以及求解时间路由能捕获多少余量。我们通过设计分析而非端到端部署来衡量这一点，从而将余量与路由器质量隔离开来。在

PolyBench/CTF-Dojo/FutureX 的 80/40/58 个任务上，我们为每个任务类别播种一个分支，在任务流中进化驾驭树，并将每个任务在所有分支上重放。随后比较 Oracle（每个任务的最佳分支）、Adapt（基于类别的路由）、Naive（仅主分支）和 Worst（每个任务的最差分支）。如图 7 所示，Adapt 在 CTF-Dojo 和 PolyBench 上均有提升，同时保留了向 Oracle 分支选择靠近的余量。在 FutureX 上，主分支优于 Adapt，因为网络源检索能力占主导地位。

要点。 驾驭专业化确实暴露了真实的适应余量，但实际路由仅捕获了其中一部分，因此将余量转化为收益是与构建分支不同的另一项挑战。

4.6 面向自动驾驶的人类转向

RQ5 探究当历史记录缺少所需的来源或访问信号时，人工引导是否有所帮助

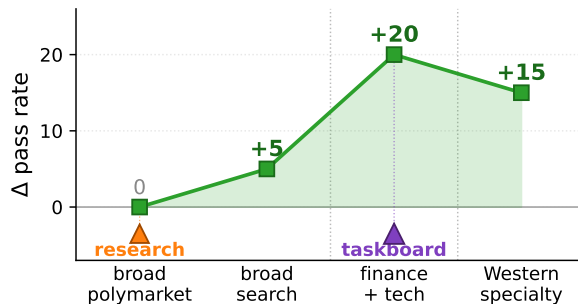


图 9：在两种人工引导机制下，FutureX 在四个任务切片上的通过率提升。橙色三角标记研究阶段引导，紫色三角标记任务板引导。提升幅度以无人环（no-HITL）运行作为基准进行衡量。

进化。由于此前所有实验均未应用人机协同（HITL）来确保公平性，我们在 100 个 FutureX 任务上对此进行评估，并将人类输入限制为两个钩子：研究阶段引导在研究受阻时提供凭证，任务板引导添加进化器无法推断的来源方向（图 8c）。图 9 展示了切片层面的效果：在广泛的预测市场问题上提升为 0，在广泛的搜索依赖问题上提升至 +5，在直接针对的金融与科技切片上达到峰值 +20，在相邻的西方专业问题上保持 +15。该模式表明，人机协同在注入缺失的外部信号时有效，而非

通用人类建议。

要点。当缺失的要素是外部来源知识而非额外的自主进化时，人类引导最为有用。

5 结论

开放式任务流为自动测试框架部署带来三项挑战：任务到达无界、任务异构以及非平稳性。自适应自动测试框架通过将持续的测试框架构建与求解时的任务适配相结合来应对这些挑战。将损失分解为演化损失与适配损失，阐明了为何单一且反复更新的测试框架并不充分：系统必须从流证据中构建缺失能力，同时为每个任务选择恰当的专用分支。实验进一步表明，三种机制是互补而非可互换的：多智能体演化构建基准特定能力，路由在分支信号可靠时利用测试框架的专用化，而人工引导则提供历史无法包含的外部信号。

6 局限性

基准覆盖范围。本文在三个开放式任务流上进行了评估：预测市场、网络安全挑战和事件预测。这些领域涵盖了无界流、任务异质性和分布非平稳性，但相同的框架还应在更多部署流上进一步测试，其中流在空间和时间上进一步扩展，以模拟真实世界的部署。

诊断损失。演化损失 L_{evo} 和适应损失 L_{adapt} 是分析量，而非直接估计的预言机损失。本文的实验通过瓶颈分析、消融和路由控制来诊断它们，而非通过预言机框架的正式估计器。

7 伦理声明

本文使用公开的研究基准任务，不引入私有用户数据。CTF-Dojo 仅在隔离的基准容器内运行，不针对真实系统。人工引导仅限于源指导、任务板编辑和凭证决策；人类不标记或暴露答案，也不选择求解器分支。

8 人工智能使用声明

我们使用人工智能助手来润色本文的写作，并在实现和评估过程中加速调试与分析。

参考文献

- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, and 1 others. 2025. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*.
- Anthropic. 2025a. Claude haiku 4.5. <https://www.anthropic.com/claude/haiku>.
- Anthropic. 2025b. Claude sonnet 4.6. <https://www.anthropic.com/claude/sonnet>.
- Shai Ben-David, John Blitzer, Koby Crammer, Alex Kulesza, Fernando Pereira, and Jennifer Wortman Vaughan. 2010. A theory of learning from different domains. *Machine learning*, 79(1):151–175.
- Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. 2020. Dark experience for general continual learning: a strong, simple baseline. *Advances in neural information processing systems*, 33:15920–15930.
- ByteDance. 2025. DeerFlow: Deep exploration and efficient research flow. <https://github.com/bytedance/deer-flow>. Open-source software, accessed 2026-05-28.
- Pu Cheng, Juncheng Liu, and Yunshen Long. 2026. Polybench: Benchmarking llm forecasting and trading capabilities on live prediction market data. *arXiv preprint arXiv:2604.14199*.
- Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario March, and Victor Lempitsky. 2016. Domain-adversarial training of neural networks. *Journal of machine learning research*, 17(59):1–35.
- Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. 1991. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.
- Seth Karten, Joel Zhang, Tersoo Upaa Jr, Ruirong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vohra. 2026. Continual harness: Online adaptation for self-improving foundation agents. *arXiv preprint arXiv:2605.09998*.

- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*.
- Jian Liang, Dapeng Hu, and Jiashi Feng. 2020. Do we really need to access the source data? source hypothesis transfer for unsupervised domain adaptation. In *International conference on machine learning*, pages 6028–6039. PMLR.
- Minhua Lin, Hanqing Lu, Zhan Shi, Bing He, Rui Mao, Zhiwei Zhang, Zongyu Wu, Xianfeng Tang, Hui Liu, Zhenwei Dai, and 1 others. 2026. Position: Agentic evolution is the path to evolving llms. *arXiv preprint arXiv:2602.00359*.
- Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, and 1 others. 2025. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*.
- Pan Lu, Bowen Chen, Sheng Liu, Rahul Thapa, Joseph Boen, and James Zou. 2025. Octotools: An agentic framework with extensible tools for complex reasoning. *arXiv preprint arXiv:2502.11271*.
- Rui Miao, Babak Shahbaba, and Annie Qu. 2025. Reinforcement learning for individual optimal policy from heterogeneous data. *Annals of statistics*, 53(4):1513.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chunliang Li, Yizhu Jiao, Kaiwen Zha, and 1 others. 2026. Skillos: Learning skill curation for self-evolving agents. *arXiv preprint arXiv:2605.06614*.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*.
- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, and 1 others. 2026. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*.
- Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. 2020. Tent: Fully test-time adaptation by entropy minimization. *arXiv preprint arXiv:2006.10726*.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- Zifeng Wang, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and 1 others. 2022a. Dualprompt: Complementary prompting for rehearsal-free continual learning. In *European conference on computer vision*, pages 631–648. Springer.
- Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. 2022b. Learning to prompt for continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 139–149.
- Z.ai. 2025. Glm-4.7. <https://www.z.ai/>.
- Zhiyuan Zeng, Jiashuo Liu, Siyuan Chen, Tianci He, Yali Liao, Yixiao Tian, Jinpeng Wang, Zaiyuan Wang, Yang Yang, Lingyue Yin, and 1 others. 2025. Futurex: An advanced live benchmark for llm agents in future prediction. *arXiv preprint arXiv:2508.11987*.
- Terry Yue Zhuo, Dingmin Wang, Hantian Ding, Varun Kumar, and Zijian Wang. 2025. Training language model agents to find vulnerabilities with ctf-doj. *arXiv preprint arXiv:2508.18370*.

基准与评估细节

在全部三个基准中，任务按时间顺序评估。求解器接收任务 x_i 时仅获得其发布时间之前的信息，而进化器仅在相应解决时间之后接收结果标签。我们的分析流水线加载每个 `instance_id` 的第一条记录，因此重试或重复日志不会改变报告的度量。三个基准共同涵盖了第 1 节中确定的三个开放式流部署维度。

A.1 多面体基准

来源与构成。

多面体基准是一个源自多市场预测市场的流，包含 2026 年 2 月 6 日至 22 日的 5,075 个任务。该流涵盖政治、体育、金融、加密货币和娱乐市场，每个任务根据官方市场结果解决。

时间顺序。对于每个市场，我们存储任务发布时间戳和结果解决时间戳。已解决的标签在相应市场解决之前对进化过程隐藏。

度量指标。设 N 为流中市场的总数，设 $\mathcal{T}$ 为已执行交易的集合，排除门控任务、空决策和跳过决策。设 $s_i \in \{0, 1\}$ 表示交易 i 是否正确， b_i 为其置信度加权投资， g_i 为其已实现利润。为 PolyBench 报告的度量指标为：

$$\begin{aligned}\text{Coverage} &= \frac{|\mathcal{T}|}{N}, \\ \text{Acc} &= 100 \cdot \frac{1}{N} \sum_{i \in \mathcal{T}} s_i, \\ \text{CWR} &= 100 \cdot \frac{\sum_{i \in \mathcal{T}} g_i}{\sum_{i \in \mathcal{T}} b_i}, \\ \text{Return} &= \text{Coverage} \cdot \text{CWR}.\end{aligned}$$

因此，准确率同时奖励广泛的覆盖范围和正确的决策。回报是一种投资组合风格的盈利能力度量：置信度加权回报（CWR）捕捉了代理实际交易的市场中每单位投资的美元加权利润，而覆盖率缩放则对仅在流的一小部分上获得的高 CWR 进行折扣。我们将回报与准确率一起报告，因为准确率平等对待每笔交易，对仓位大小不敏感，而在置信度加权投资下，一笔自信的错误交易可能抵消几笔自信的正确交易；因此，回报反映的是已实现的盈亏，而不仅仅是方向正确性。

A.2 CTF-Dojo

来源与构成。CTF-Dojo 是一个包含 261 个挑战的安全竞赛流，取自 `pwncollege/ctf-archive`，按时间顺序从 2011 年到 2024 年排列。挑战涵盖二进制利用、网络安全、密码学、逆向工程和取证，揭示了挑战风格和工具随时间的演变。

沙箱与验证。每个挑战在具有受限网络策略的每个任务 Docker 沙箱中运行。旗帜以文本形式提交，并通过与官方旗帜的 SHA-256 哈希比较进行验证。

度量指标。本文报告了基准预算内所解决挑战的百分比，即一次通过率（Pass@1）。所有夺旗道场（CTF-Dojo）的汇总结果均采用与发布流相同的时间顺序。

A.3 未来未知（FutureX）

来源与构成。未来未知（FutureX）是一个包含 503 个问题的预测流，时间跨度为 2026 年 1 月至 4 月，共 82 天，源自未来未知 - 过去（FutureX-Past）。问题涵盖金融、科技、地缘政治和娱乐领域，并包含中英文两种语言版本。其中，中文金融切片要求超越默认的纯英文检索进行来源发现。

时间检索与过滤。未来未知（FutureX）任务具有历史性，但网页和搜索索引在事件发生后仍会持续变化。为避免标签泄漏，每个任务均根据其时间元数据设定截止日期进行求解。在严格的内置检索中，维基百科内容通过修订版应用程序编程接口（API）获取截止日期前的最新修订版本，鸭鸭走（DuckDuckGo）搜索结果通过从超文本标记语言日期（`htmldate`）和统一资源定位符（URL）模式中提取的发布日期进行过滤，结构化经济序列则通过观测和实时端点查询并截断至截止日期。对于通过沙箱执行的演化工具，实时命令输出在求解器看到之前会经过大语言模型（LLM）时间过滤器处理：过滤器接收任务、截止日期和检索内容，然后返回“干净”（CLEAN）或将截止日期后的值、行和片段替换为“[已编辑]（[REDACTED]）”。这既保留了截止日期前的证据，又阻止了会揭示已解决答案的动态网页内容。

度量指标。本文报告了官方未来未知（FutureX）标准下的一次通过率（Pass@1）。切片级人类引导分析采用相同的通过标准，并按图 9 所示的分组对任务进行汇总。

B 实现与可复现性细节

执行协议。所有主要运行均使用提供商托管的 LLM 应用程序编程接口，具备原生工具调用能力，并采用基准测试所用的相同时间顺序任务顺序。报告的数字由相应的 results.jsonl 文件生成。闭源提供商托管模型不公开参数数量；因此，我们报告评估的任务数量和进化周期，而非 GPU 小时数。完整系统运行包含

PolyBench/CTF-Dojo/FutureX 各自的 5,075/261/503 条求解轨迹和 51/14/26 个进化周期。

计算与资源。所有模型推理均通过提供商托管的应用程序编程接口执行；主要的自适应自动测试框架运行使用 Claude Sonnet 4.6 进行求解，使用 Claude Opus 4.6 进行进化，其他提供商托管模型仅用于相应的基线行。我们不训练或微调模型权重，也不使用本地 GPU 计算进行模型优化。本地计算用于编排、结果聚合、图表生成以及基于 Docker 的基准测试执行，包括每个任务的 CTF-Dojo 和 FutureX 沙箱。由于所评估的闭源模型不公开参数数量，任务数量和进化周期是主要的计算描述指标。超参数。表 3 列出了所有主要运行所使用的各基准测试超参数。由于全程温度为零，报告指标为点估计而非重复采样的平均值。

种子测试框架。表 4 报告了在任何周期运行之前进化器所继承的内容。种子提示刻意保持紧凑：PolyBench 和 CTF-Dojo 的种子提示分别为 27 行和 24 行，FutureX 较长，因为它记录了时间检索契约。所有三个基准测试均以零种子技能、工具和记忆条目开始，因此除种子提示和 FutureX 基础设施脚手架之外的任何测试框架组件均由进化器自行构建；这将主要结果中报告的增益与种子的事先手工工程隔离开来。

令牌成本与墙钟时间。表 5 总结了五个最相关系统的每系统令牌使用量和墙钟时间。求解器令牌由 results.jsonl 中的每任务字段求和得出；进化器侧令牌被省略，因为编排器未将其持久化在

表 3：自适应自动测试框架在三个基准测试中的超参数。EGL 是期望学习增益触发器，用于决定是否运行周期。所有运行对求解器和进化器均使用 $T = 0$ ，以将增益归因于算法而非采样噪声。

Hyperparameter	PolyBench	CTF-Dojo	FutureX
<i>Models</i>			
Solver	Sonnet 4.6	Sonnet 4.6	Sonnet 4.6
Evolver	Opus 4.6	Opus 4.6	Opus 4.6
Router	Sonnet 4.6	Sonnet 4.6	Sonnet 4.6
<i>Sampling & budget</i>			
Solver temperature	0.0	0.0	0.0
Evolver temperature	0.0	0.0	0.0
Solver max turns	80	80	80
Evolver max tokens	128k	128k	128k
<i>Stream & schedule</i>			
Total tasks	5,075	261	503
Batch size	100	20	20
Evolution cycles	51	14	26
EGL threshold	0.05	0.05	0.05
EGL window	3	3	3
Solve workers	24	8	10
<i>Multi-agent & routing</i>			
Research parallel agents	3	3	3
Build/verify retries	3	3	3
Routing confidence threshold	0.7	0.7	0.7
<i>Sandbox</i>			
Solver sandbox network	none	none	bridge
Evolver sandbox network	none	none	bridge

表 4：在任何循环运行之前发送给进化器的种子测试框架。技能统计顶级技能目录；记忆统计 JSONL 条目；基础设施指示种子是否包含基础设施目录。

Benchmark	Prompt LOC	Skills	Tools	Memory	Infra
PolyBench	27	0	0	0	no
CTF-Dojo	24	0	0	0	no
FutureX	114	0	0	0	yes

已发布的工件。墙钟时间是每个任务经过秒数的总和，不包括编排开销。

时间揭示。每个任务存储一个发布时间戳，并在可用时存储一个解决时间戳。求解器调用根据发布时间进行过滤。进化循环立即接收轨迹，但仅在任务解决后才接收结果反馈，因此未解决的任务保持为未标记的历史记录，而不是泄露的监督信息。

工作区工件。进化器工作区在循环之间持久保存任务板、研究日志、验证器笔记、测试和架构笔记。这些工件与求解器工作区分开：进化器可以在进化期间更新测试框架文件，而求解时的路由器仅检查分支元数据并为传入任务选择分支。

分支重放协议。用于路由分析

表 5：各系统的求解器令牌成本与墙钟时间。令牌数由每个任务的 `input_tokens / output_tokens` 在 `results.jsonl` 中求和得到。墙钟时间对每个任务的已用秒数求和，并排除编排开销。在发布的工件中，编排器未持久化进化器侧的令牌。

System	Bench	In (M tok)	Out (M tok)	Hours	Tasks/h
Sonnet (no-evo)	PolyBench	35.3	5.0	25.6	198.3
	CTF-Dojo	138.4	2.1	22.4	11.7
	FutureX	55.5	0.7	34.2	14.7
A-Evolve	PolyBench	365.4	4.2	23.9	212.3
	CTF-Dojo	111.3	1.7	11.4	23.0
	FutureX	268.8	1.2	13.8	36.5
Meta-Harness	PolyBench	245.0	9.6	49.7	102.1
	CTF-Dojo	142.7	2.2	20.0	13.0
	FutureX	39.1	0.5	7.9	63.8
Multi-agent	PolyBench	264.2	14.5	78.0	65.1
	CTF-Dojo	180.2	2.5	21.2	12.3
	FutureX	130.7	1.0	12.1	41.7
Full System	PolyBench	233.2	12.2	59.5	85.2
	CTF-Dojo	169.0	2.4	21.1	12.4
	FutureX	25.6	0.5	6.6	75.7

在附录 G 中，进化测试框架树中的每个分支都在一个精选的任务子集上重放，以构建 Oracle 和 Worst 对照。Oracle 和 Worst 是事后诊断边界；部署的路由器仅看到任务上下文和分支元数据，而非标签或分支结果。

人工引导记录。人工引导事件是作者提供的系统干预，而非招募的人类受试者标注。每个事件都记录了触发阶段、请求的外部信号以及记录响应的工作区位置；表 13 复现了完整的事件日志。这使得引导可审计，并将人工输入保持为来源或访问指导，而非答案标签。

C 基准非平稳性诊断

图 12–14 为三个流的时间结构提供了描述性诊断。这些图不用作评估度量；相反，它们展示了为什么这些基准不是静态的独立同分布池，以及为什么拟合于早期观测的测试框架可能与后续任务不匹配。大多数面板由任务元数据、任务文本或基准侧属性计算得出；使用结果或基线求解器的面板仅作为描述性可解性代理。

PolyBench。预测市场在评估期间在难度和可交易性两方面都发生了变化（图 12）。早期市场更常具有流动性且已经明确：可交易市场的比例——

市场价从早期的 97% 骤降至后期的 31%，最高价超过 0.95 的比例从 44% 降至 29%。与此同时，接近均衡的市场从 18% 增至 35%，意味着后期任务更常需要超越简单跟随强市场共识的证据。市场价格正确性代理指标也随时间变化，从早期的 84% 降至后期的 77%。这些变化共同使得固定的预测市场策略变得脆弱：校准、弃权与证据收集必须随着数据流从流动性强且明确的市场转向更稀疏且模糊的市场而调整。

CTF-Dojo。CTF-Dojo 展现出另一种形式的非平稳性：数据流扩展至早期历史中未出现的竞赛与挑战惯例（图 13）。源竞赛的累计数量按时间顺序持续增加，到后期数据流中，来自前三分之一未见竞赛的任务比例达到 100%。50 任务窗口内所代表的竞赛数量也大幅波动，因此相邻任务可能需要对文件布局、评分惯例及预期利用风格做出不同假设。跨竞赛得分变异系数进一步表明，竞赛并非可互换的池子；每个赛事对难度和挑战设计的校准可能不同。这促使持续构建可复用的安全技能，但也警示不应将早期 CTF 经验视为统一可迁移。

FutureX。FutureX 在来源、语言和难度维度上发生变化（图 14）。批次级基线准确率从 20% 到 80% 不等，表明按时间划分的批次在可解性上差异显著。后期批次包含更多中文标题问题及更多与难以直接搜索的平台相关的问题，而较难的 3–4 级问题占比在同一区域急剧上升。中文答案要求也主要出现在后期批次中。这些变化解释了为何 FutureX 同时强调构建与适应：框架必须获得更好的来源查找与时间检索行为，而解题时的路由必须选择适合任务语言、来源和难度特征的分支。

D 进一步实验

我们纳入两项补充诊断，以阐明主要收益的来源，而不重复

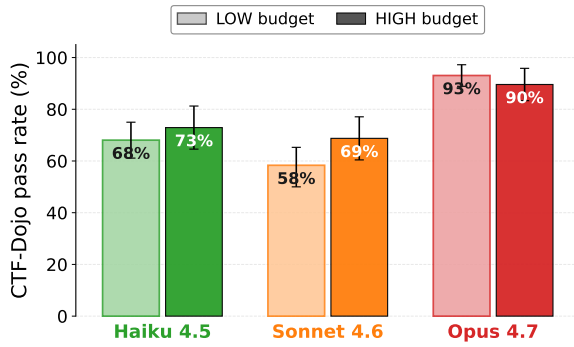


图 10：进化器在 CTF-Dojo 上的能力。通过率随进化器模型增强而提升；更高的预算对 Haiku 和 Sonnet 有帮助，但一旦 Opus 已达到高性能，额外预算带来的增益甚微。

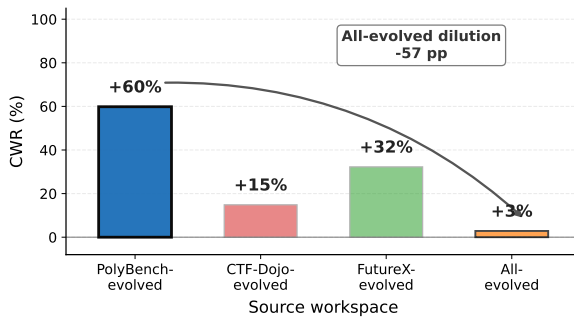


图 11：PolyBench 工作区稀释。一个 PolyBench 进化的工作区达到最高的 CWR，而合并所有进化的工作区则急剧降低 CWR，这支持了需要专门化 harness 分支而非单一密集 harness 的观点。

在第 4 节中进行了 RQ 分析。进化器能力与构建预算。图 10 在 CTF-Dojo 上变化进化器模型和构建预算。更强的进化器实现更高的通过率，而额外预算主要帮助较弱的进化器，对最强模型则趋于饱和。跨域工作区稀释。图 11 比较了在不同域上进化的工作区在 PolyBench 上的 CWR。PolyBench 特定工作区表现最佳，而全进化工作区损失了 57 个 CWR 点，表明混合异构经验会稀释与域相关的 harness 结构。

E 按域和按类别细分

CTF-Dojo。 表 6 按类别报告了 Pass@1。完整系统在 web (+27（相对于 Sonnet）和 crypto (+19); binary/pwn 上增益最大，仍然

表 6：CTF-Dojo 上按类别的 Pass@1 (%)。类别从 detail 字段解析；binary/pwn 合并了传统上等价的 CTF 标签。粗体标记每行最佳系统。

Category	N	Sonnet	A-Evolve	Meta-H.	Multi	Adapt.	Full
crypto	74	52.7	66.1	56.2	67.6	55.6	72.1
binary/pwn	41	4.9	11.5	2.5	7.0	4.8	14.8
web	11	45.5	41.7	41.7	33.3	50.0	72.7
reverse	65	49.2	62.7	48.7	59.7	57.0	66.2
forensics	12	58.3	61.5	64.3	64.3	71.4	58.3
misc	58	20.7	23.5	29.5	34.1	40.5	25.6
Overall	261	37.2	45.2	41.0	47.9	46.0	50.2

表 7：FutureX 上按切片的 Pass@1 (%)。语言通过问题中的中文字符检测；域通过关键词匹配推断（other 捕获未匹配的问题）。N 来自 Sonnet 基线运行。zh, geopolitics 行包含单个任务，仅为完整性而报告。粗体标记每行最佳系统。

Lang	Domain	N	Sonnet	A-Evolve	Meta-H.	Multi	Adapt.	Full
en	finance	76	21.1	53.9	21.1	52.6	43.4	56.6
en	tech	8	25.0	50.0	37.5	50.0	37.5	50.0
en	geopolitics	43	41.9	62.8	37.2	62.8	60.5	65.1
en	sports	62	38.7	58.1	37.1	59.7	56.5	54.8
en	entertainment	37	27.0	43.2	21.6	40.5	48.6	37.8
en	other	219	38.8	52.1	37.4	55.7	48.9	51.1
zh	finance	10	0.0	0.0	0.0	30.0	0.0	20.0
zh	geopolitics	1	100	100	0	0	0	0
zh	entertainment	25	0.0	0.0	0.0	0.0	0.0	0.0
zh	other	22	0.0	0.0	0.0	4.5	0.0	4.5
Overall		503	31.0	47.5	29.4	49.5	44.1	47.3

即使在进化和路由之后，最难的类别仍为 14.8%，这与第 4.3 节中确定的沙箱载荷处理瓶颈一致。

FutureX。 表 7 按语言和推断领域报告了首次通过率 (Pass@1)。英语切片从进化中获益最多；小型中文金融切片揭示了来源发现瓶颈，即当平台处于搜索墙之后时，仅靠进化或路由都无法恢复。领域通过问题文本的关键词匹配推断，因为未来 X (**FutureX**) 结果不存储官方领域标签；因此“其他”桶聚合了未匹配的问题。

PolyBench。 表 8 按推断类别报告了准确率/回报。体育在组合比率中占主导地位，因为流动性强的体育市场承载了大部分美元加权利润；政治尽管准确率高，但回报几乎为零。类别通过轨迹提示的事件描述的关键词匹配推断。

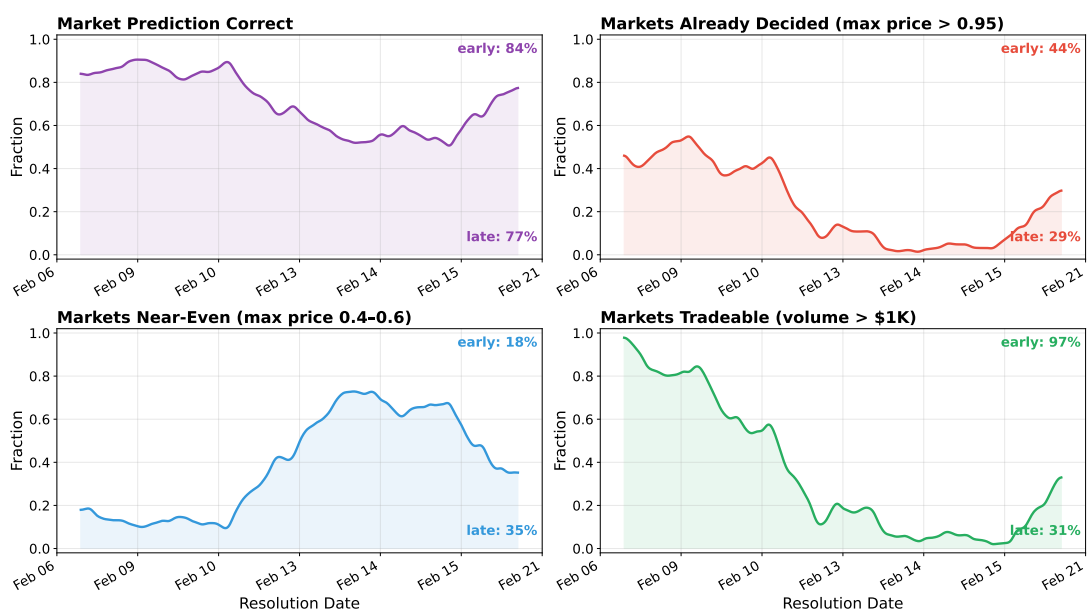


图 12: 多基准 (PolyBench) 非平稳性。市场难度和可交易性随时间变化：后期市场较少具有决定性或不具流动性，更多接近均势。

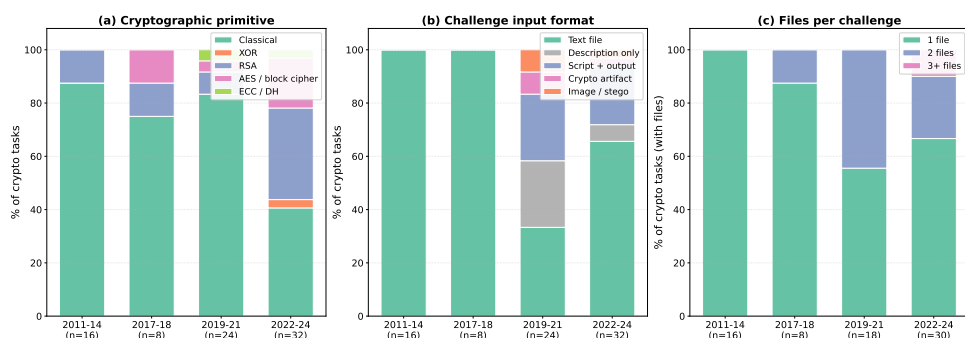


图 13: 夺旗道场 (CTF-Dojo) 非平稳性。按时间顺序的流不断引入新竞赛并增加跨竞赛变异性，因此早期挑战经验并非均匀可迁移。

F 多智能体进化动力学 表 9 报告了全流系统对比（无进化 → 单智能体 → 多智能体）以及多智能体运行的时间，补充而非重复图 6。该图在精心挑选的样本子集上运行四阶段进化器，对比无记忆和无反馈消融；此处的表则报告每个系统在整个基准上的主要度量，与表 2 的多智能体列一致。新增的独特信号是时间：峰值列给出了多智能体运行累积均值最高的周期索引，峰值远早于最终周期与图 1 中的过拟合趋势一致。在多基准上，多智能体峰值出现在 51 个周期中的第 22 个周期，在未来 X 上出现在 26 个周期中的第 10 个周期，而夺旗道场在所有 14 个周期中持续累积效用。

G 路由行为与分支性能

本附录扩展了背后的路由分析

图 7: 我们在每个基准上使用两个相关但不同的子集：(i) 导航运行子集 (CTF-Dojo: 60 个任务, PolyBench: 100 个, FutureX: 80 个)，在该子集上大语言模型路由器被实时部署，我们观察其分支分配情况；(ii) 重放子集 (CTF-Dojo: 40 个, PolyBench: 80 个, FutureX: 58 个)，这是相同任务的一个子集，其中每个分支都已端到端重放，因此每个任务都带有完整的跨场地得分向量。重放子集必然更小，因为某些早期周期的分支对于第 1 批任务并不存在。从重放子集派生出四个逐任务序列：Oracle = 最佳场地，Adapt = 大语言模型路由器在导航运行中的实际选择，Naive = 固定主场（无分支），以及 Worst = 最差场地。

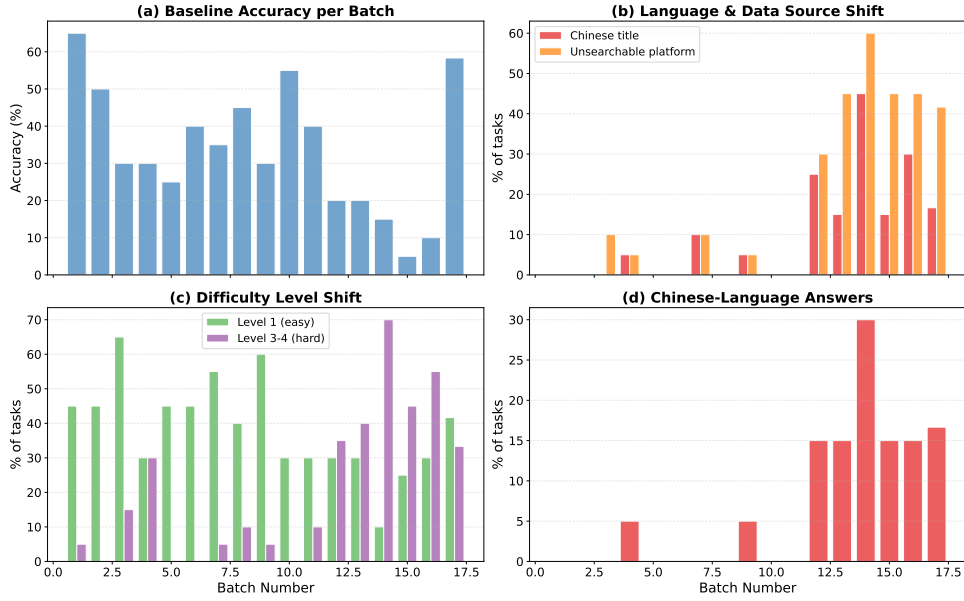


图 14: FutureX 非平稳性。语言、来源可访问性、难度和答案格式在不同批次间发生变化，当对所有任务重复使用一个静态测试框架时，会产生求解时间框架不匹配。

表 8: PolyBench 各类别度量。类别通过轨迹提示中事件描述的关键词匹配推断；其他类别捕获未匹配的市场。每个单元格报告准确率（%）/ 回报率（%）。粗体标记每行准确率最佳的系统。

领域	N	Sonnet	A-Evolve	元框架	多智能体	自适应	完整系统
politics	372	18.3/-5	11.3/+1	71.8/-2	87.9/-3	87.6/-2	87.1/-4
sports	1,120	23.8/+3	16.4/+8	55.8/+594	81.7/+659	80.0/+642	84.6/+596
finance	240	19.6/+2	18.3/+8	64.6/+7	81.7/+10	77.1/+8	80.4/+9
crypto	447	20.4/-3	17.7/+3	66.9/+2	89.3/+4	88.4/+4	90.8/+5
entertainment	218	22.0/-2	12.8/+1	64.7/+95	89.4/+105	87.6/+101	89.0/+83
other	2,678	22.6/+3	20.7/+9	40.8/+356	75.4/+394	72.3/+409	76.2/+374
Overall	5,075	22.2/+2	18.4/+7	50.8/+320	79.8/+351	77.4/+352	80.9/+330

表 9: 全流程系统对比，补充图 6 中基于子集的消融。No-evo 是无进化的 Sonnet 基线；Single 是 A-Evolve（单智能体进化器）；Multi 是四阶段多智能体进化器。Peak 列报告多智能体运行累计均值最高的周期索引，Cycles 是进化周期的总数。

Benchmark	Metric	No-evo	Single	Multi	Peak	Cycles
PolyBench	Acc	22.2	18.4	79.8	22	51
CTF-Dojo	Pass@1	37.2	45.2	47.9	1	14
FutureX	Pass@1	31.0	47.5	49.5	10	26

路由器将每个任务发送到何处。表 10 报告了导航运行子集上每个分支的路由量和通过率。路由器提示（附录 J）确实允许在没有分支强匹配时回退到主场，但在这些子集上，路由器总是识别出特定机制的分支

并且从不调用该回退。实现通过率低的分支（例如 CTF-Dojo 上的 branch/pwn，FutureX 上的 branch/lvl3）本身并不是失败的分支——路由器将真正困难的任务发送给它们，并且相应任务在重放子集上的 Oracle 通过率也很低（表 11）。路由恢复了多少自适应能力

差距。

表 11 给出了在重放子集上的 Oracle/Adapt/Naive/Worst 总体比较，包含 95% 自助置信区间和经 Holm – Bonferroni 校正的配对 Wilcoxon p 值。Oracle – Naive 差距，即我们对适应损失 L_{adapt} 的经验估计，在 CTF-Dojo (+37.5 pp, $p_{adj} = 4.8 \times 10^{-4}$) 和 PolyBench 上较大且显著 (+8.8pp CWR, $p_{adj} = 1.8 \times 10^{-5}$ ；在 FutureX 上差距较小且校正后不显著，这与 § 4.3 的发现一致，即源获取而非分支选择是制约因素

表 10: RQ4 导航分析子集上的各分支路由量 (§ 4.5)。每一行报告在 Adapt 条件下 LLM 路由器实际发送到该分支的任务, 以及相应的 Pass@1 (CTF-Dojo、FutureX) 或交易市场中的命中率 (PolyBench)。在这些子集上, 路由器从未调用其提示允许的主回退 (附录 J)。

Bench	Branch	N routed	Pass / HitRate (%)
<i>CTF-Dojo (60 tasks across 3 batches; per-task best-of-7 venues)</i>			
	branch/crypto	20	50.0
	branch/rev	20	30.0
	branch/pwn	12	0.0
	branch/misc	4	75.0
	branch/web	2	0.0
	branch/forensics	2	50.0
<i>PolyBench (100 tasks across 4 batches; per-task best-of-5 venues)</i>			
	branch/sports	71	67.6
	branch/finance	14	21.4
	branch/culture	12	33.3
	branch/politics-world	3	66.7
<i>FutureX (80 tasks across 3 batches; per-task best-of-5 venues)</i>			
	branch/lvl1	28	53.6
	branch/lvl2	30	30.0
	branch/lvl3	8	0.0
	branch/lvl4	14	14.3

该能力。Adapt 在 CTF-Dojo 和 PolyBench 上弥合了差距的相当一部分, 但在 FutureX 上略逊于 Naive, 再次反映了源获取瓶颈。

逐批次视图。 表 12 报告了同一系列在重放子集上按批次拆分的结果 (仅显示具有完整跨分支重放的批次, 因此例如 CTF-Dojo 仅包含批次 2 和 3)。CTF-Dojo 的分支树在这两个批次之间有所改善 (Oracle 45 $\rightarrow$ 65 %), 因为新的专门化开始生效; PolyBench 显示批次 4 表现平稳, 随后在批次 5 中由体育相关任务带动恢复; FutureX 的批次 3 具有最大的 Oracle – Adapt 提升空间, 其中路由器的基于层级的分支在同一任务上不如主分支可靠。

H 人在回路事件日志

表 13 报告了来自 FutureX RQ5 运行的完整人在回路事件日志 (§ 4.6)。该运行使用精心设计的 5 批次 FutureX 流 (设计 C: 100 个任务, 5×20), 包含两个工程化的机制转变, 使用 Sonnet 4.6 求解器、Opus 4.6 进化器, 以及 hitl_enabled=true。分析师和构建者决定何时调用每个钩子; 人类通过 Telegram 根据预先编写的备忘单进行响应。两个 P2 (研究阶段) 事件在第 1 周期触发以引导搜索管道, 一个实质性的 P3 (任务板) 事件在第 3 周期触发以引导进化器转向西方和中国特色菜。

表 11: 图 7 的数值对照。在 RQ4 子集上基于重放的 Oracle/Adapt/Naive/Worst 比较 (§ 4.5)。Oracle 是每个任务的最佳场所; Adapt 是 LLM 路由器的选择; Naive 是固定的主场 (无分支); Worst 是每个任务的最差场所。CTF-Dojo 和 FutureX 使用 Pass@1 (%) ; PolyBench 使用 CWR (%)。均值报告 95% 自助置信区间; 差距使用配对单侧 Wilcoxon 符号秩检验, 并进行 Holm – Bonferroni 校正 p 值。Oracle – Naive 差距是经验适应损失 L_{adapt} 。

	CTF-Dojo (Pass%)	PolyBench (CWR%)	FutureX (Pass%)
N tasks	40	80	58
Oracle	55.0 [40.0, 70.0]	+12.0 [+2.1, +21.2]	46.6 [32.8, 60.3]
Adapt	35.0 [20.0, 50.0]	+5.9 [−6.6, +17.4]	34.5 [22.4, 46.6]
Naive	17.5 [7.5, 30.0]	+3.2 [−7.5, +13.0]	39.7 [27.6, 51.7]
Worst	7.5 [0.0, 17.5]	−10.6 [−25.9, +4.0]	22.4 [12.1, 32.8]
L_{adapt} (Oracle – Naive)	+37.5	+8.8	+6.9
Adapt – Naive	+17.5	+2.7	−5.2

表 12: RQ4 路由子集上 Oracle/Adapt/Naive/Worst 均值的按批次分解。CTF-Dojo 和 FutureX 使用 Pass@1 (%) ; PolyBench 使用 CWR (%)。Adapt 在 CTF-Dojo 上最一致地缩小了 Oracle – Naive 差距, 因为分支质量稳定; 在 FutureX 上差距较小且噪声较大, 因为源获取而非分支选择主导了失败模式。

Benchmark	Batch	N	Oracle	Adapt	Naive	Worst
CTF-Dojo (Pass%)	2	20	45.0	30.0	15.0	10.0
	3	20	65.0	40.0	20.0	5.0
PolyBench (CWR%)	2	20	+16.7	+11.6	+7.8	−100.0
	3	20	+13.0	+7.8	+1.9	−8.9
	4	20	+1.1	+1.1	+1.4	−5.1
	5	20	+14.7	+5.2	+0.5	−4.7
FutureX (Pass%)	2	20	45.0	40.0	45.0	40.0
	3	19	57.9	26.3	52.6	21.1
	4	19	36.8	36.8	21.1	5.3

端点。剩余的 P3 提示 (第 1、2、4 和 5 周期) 返回跳过, 符合备忘单协议, 即人类仅在备忘单有相关条目时才进行干预。每个机制上的切片级提升 (机制 1 – 5 上的 0, +5, +20, +15, 0) 报告在图 9 中。

I 运行细节分析

表 14 报告了每个任务的轮次计数和经过秒数, 补充了表 5 中的总成本摘要。在 CTF-Dojo 和 FutureX 上, 两种分布均呈右偏, 少数长时间运行的任务将均值拉高至中位数之上; 因此我们同时报告两者。CTF-Dojo 的求解器预算始终饱和——Sonnet 的平均 89.4 轮反映了大量任务循环直至达到上限, 而进化变体通常

表 13: 来自 FutureX RQ5 运行的完整人在回路事件日志（设计 C, 5 批 $\times$ 20 个任务）。P2（凭证）是研究阶段的凭证钩子；P3（看板）是任务板引导钩子。两个 P2 事件在第 1 周期触发以引导搜索流水线；一个实质性 P3 事件在第 3 周期触发，引导进化器朝向西方和中国专业端点。其余 P3 提示返回跳过，与备忘单协议一致。通过 Telegram 提供的 API 令牌已脱敏。

周期 钩子 键 触发上下文 人类响应				
1	P2 (cred.)	EXA_API_KEY	Phase-2 research needs Exa search API.	[REDACTED] (key supplied)
1	P2 (cred.)	SERPER_API_KEY	Phase-2 research needs Serper Google API.	[REDACTED] (key supplied)
1	P3 (board)	cycle1	7 tasks fail: solver has no web search tool.	skip
2	P3 (board)	cycle2	9 tasks fail: search-pipeline entry never invokes search functions.	skip
3	P3 (board)	cycle3	deterministic fallback used; structured-data API gap.	“Specialty data tasks need direct endpoint integrations beyond generic web search. Build skills/tools for Western (US equities OHLC, Box Office Mojo, . . .) and Chinese (Eastmoney secid, Maoyan film board, Kol-Rank, . . .) endpoints.”
4	P3 (board)	cycle4	17 tasks fail: deterministic fallback used.	skip
5	P3 (board)	cycle5	20 tasks fail: Chinese niche-ranking API absent.	skip

通常更早达到标志（或放弃）。PolyBench 以直接决策为主：大多数任务立即提交（轮次计数为 1），因为提示中已包含求解器所需的市场上下文。

表 14: 跨系统和基准的每个任务求解器轮次和墙钟秒数。一轮是一次工具调用, 包括最终提交; 因此, 直接提交且无其他工具使用的任务计为 1。轮次和秒为算术平均值; 添加中位数列是因为在 CTF-Dojo 和 FutureX 上两种分布均呈右偏。墙钟时间不包括编排开销。

System	PolyBench				CTF-Dojo				FutureX			
	$\overline{\text{turns}}$	med	$\overline{\text{sec}}$	med	$\overline{\text{turns}}$	med	$\overline{\text{sec}}$	med	$\overline{\text{turns}}$	med	$\overline{\text{sec}}$	med
Sonnet	1.0	1.0	18.2	18.1	89.4	48.0	308.4	282.8	13.4	16.0	244.4	271.4
A-Evolve	1.7	2.0	17.0	15.7	17.8	10.0	156.7	76.9	16.9	6.0	98.8	41.9
GEPA	2.1	2.0	30.4	22.5	50.1	33.0	250.3	219.7	0.8	1.0	4.6	4.6
Meta-Harness	2.4	2.0	35.2	29.5	51.3	39.0	275.9	244.4	4.6	4.0	56.4	50.4
Continual H.	2.3	2.0	27.4	21.6	15.3	12.0	115.3	82.3	6.8	5.0	167.8	112.7
SkillOS	2.7	2.0	32.1	23.0	31.4	16.0	224.2	161.0	5.3	4.0	120.3	84.1
OctoTools	5.2	4.0	68.6	63.5	44.1	35.0	264.8	253.7	1.0	1.0	10.7	10.5
Multi-agent	5.7	6.0	55.3	48.7	40.1	35.0	293.1	245.2	11.3	3.0	86.4	36.2
Adaptive	2.8	3.0	43.7	38.3	29.7	25.0	281.9	238.1	10.1	8.0	74.5	55.7
Full System	5.4	6.0	42.2	40.5	36.4	42.0	290.9	255.9	4.2	4.0	47.6	38.6

J 系统提示

我们逐字复现自适应自动测试框架（Adaptive Auto-Harness）使用的系统提示，与智能体接收到的完全一致。花括号占位符如{benchmark_context}, {regime}、{workspace_extras},和{categories}由框架在运行时按基准或制度替换；我们保留它们以显示模板结构。

Solve-time router

你是一个快速任务路由器。对于每个任务，你必须从提供的列表中准确选择一个分支。

Decision procedure:

1. Read each branch's README, focusing on its "When to route here" section.
2. Match the task description against each branch's positive signals (categories, keywords, metadata) and negative signals (NOT applicable).
3. If exactly one specialized branch matches strongly → route there with high confidence (≥ 0.7).
4. If multiple branches partially match or none match clearly → route to "main" with confidence reflecting your uncertainty.
5. Branches without a clear "When to route here" section should be treated as low-confidence candidates – prefer main unless the regime is obvious.

Output JSON only:

```
{"branch": "main", "confidence": 0.8, "reason": "brief reason"}
```

Researcher

You are a research agent – PHASE 2 of 4 in the evolution cycle.

PHASE SEQUENCE:

1. ANALYZE → analyst identified gaps, wrote task_board.md
2. RESEARCH (you) → discover solutions for your assigned regime
3. BUILD → builder reads YOUR research to write code
4. VERIFY → verifier tests what the builder created

上游：分析师将你分配到制度“{regime}”，因为该区域的任务正在失败。

DOWNSTREAM: The builder will read your research records to decide what code to write. For each approach you test, document:

- How to implement it (endpoint, arguments, response format)
- What it covers and what it doesn't
- Whether it works reliably (tested with real calls)

研究方法:

1. Read /solver_workspace/ to see what's already implemented
2. Read /evolver_workspace/research_log.jsonl to avoid retesting
3. Search the web for solutions, libraries, APIs, reference code
4. Test NEW approaches that complement what already exists

You can utilize Full network access if that's available.

Write findings to /evolver_workspace/tests/research_{regime}.jsonl.

WORKSPACE LAYOUT:

- | | |
|---------------------|--|
| /solver_workspace/ | – solver workspace (read current code) |
| /evolver_workspace/ | – evolution state |
| research_log.jsonl | – existing records (read to avoid retesting) |
| tests/ | – write findings HERE |
| /trajectories/ | – READ-ONLY solver conversations per task |

BASH OUTPUT IS CAPPED AT 100 KB PER CALL (first 50 KB + last 50 KB, middle elided). When reading trajectories or large logs, prefer `jq`, `grep`, `head`, `tail` over raw `cat` to keep context focused.

{benchmark_context}

Builder

You are a builder – PHASE 3 of 4 in the evolution cycle.

PHASE SEQUENCE:

1. ANALYZE → analyst identified failure patterns in task_board.md
2. RESEARCH → agents discovered solutions in research_log.jsonl
3. BUILD (you) → implement solutions from verified research
4. VERIFY → verifier tests YOUR code

上游: 阅读 research_log.jsonl 以获取已验证的方法 (works=true)。task_board 告诉你哪些能力最为重要。架构文档展示了已构建的内容。

DOWNSTREAM: The verifier will test your code. If verification fails, you get the report and can retry (max 3 attempts).

WORKSPACE LAYOUT:

/solver_workspace/	– the solver's workspace
prompts/system.md	– solver prompt (update if needed)
tools/	– evolved tool scripts
infra/	– evolved infrastructure (if applicable)
{workspace_extras} /evolver_workspace/	
task_board.md	– failure regimes from analyst
research_log.jsonl	– verified approaches from research
architecture.md	– UPDATE with what you built
/trajectories/	– READ-ONLY solver conversations

首先阅读现有代码。进行扩展，而非重写。不要运行 git
——框架会处理提交。

BASH OUTPUT IS CAPPED AT 100 KB PER CALL (first 50 KB + last 50 KB, middle elided). When scanning large files, prefer `jq`, `grep`, `head`, `tail` over raw `cat`.

{benchmark_context}

Verifier

You are a verification agent – PHASE 4 of 4 in the evolution cycle.

PHASE SEQUENCE:

1. ANALYZE → analyst identified failure patterns
2. RESEARCH → agents discovered solutions
3. BUILD → builder wrote code implementing those solutions
4. VERIFY (you) → test the builder's code before it goes live

UPSTREAM: The builder just wrote/modified code in /solver_workspace/. Your job is to verify it works before the next batch.

下游: 如果通过，代码将上线。如果失败，构建者会收到你的报告并重试。请具体说明失败原因。

WORKSPACE LAYOUT:

/solver_workspace/	– solver workspace (test code here)
/evolver_workspace/	– evolution state
task_board.md	– context for test cases
tests/	– write your test scripts HERE
/trajectories/	– READ-ONLY solver conversations

Output: VERDICT: PASS or FAIL, then list each test with its result.

BASH OUTPUT IS CAPPED AT 100 KB PER CALL (first 50 KB + last 50 KB, middle elided). Prefer `jq`, `grep`, `head`, `tail` over raw `cat` when inspecting large files.

{benchmark_context}

Analyst

You are a failure analyst – PHASE 1 of 4 in the evolution cycle.

PHASE SEQUENCE:

1. ANALYZE (you) → write task_board.md with failure patterns + priorities + TARGET
2. RESEARCH → agents investigate top-K gaps from YOUR task board
3. BUILD → builder implements solutions PER TARGET (main first, then branches)
4. VERIFY → verifier tests what the builder created per target

你的任务板直接驱动接下来研究和构建的内容。请具体说明缺少哪种能力——模糊的差距会导致研究缺乏重点。

该系统使用导航：git 分支隔离求解器策略。每个任务在求解前被路由到最佳分支。你的职责包括决定每个修复应落在何处。

CRITICAL: TRANSFERABILITY ANALYSIS (do this BEFORE writing the task board)

Naive evolution accumulates "shortcut artifacts" – skills, tools, prompt rules, or memory entries that helped one batch but break later tasks with different distribution. Examples seen in past runs:

- A "stop searching at 6 queries" rule helped batch 7 (sports) at 95% but crashed batches 14-20 (Chinese music) to 15% because those tasks needed longer searches.
- A `bls\_forger.md` skill with 40s overhead and hard-coded BLS curve parameters helped 1 BLS task but slowed all other crypto tasks.
- 22 hard-coded "wrong flag" entries in system.md memorized past failures without abstracting the underlying lesson, bloating the prompt.

你在第一阶段的任务是在提出更多修复之前，审计演化状态中不可转移的工件。使用 bash 来：

1. Compare per-CATEGORY pass rates ACROSS cycles (read multiple trajectories/batch_NNNN/index.txt files). If category X had 80% in early batches and 30% in recent batches – that's degradation.
2. List recent additions to skills/, tools/, prompts/system.md, memory/ and ask: which CATEGORIES does each artifact help vs. hurt?
 - Look for hard-coded task IDs, year-specific logic, single-domain rules
 - Look for prompt rules added recently that contradict older rules
3. Check the strategy_tree.md routing stats: if a branch is dragging overall performance down (worse than main on its routed tasks), mark it for retirement.

For each non-transferable artifact found, add a "## Toxic Artifacts" section to the task board listing:

- artifact_name: helps {categories} but hurts {categories} → ACTION: move-to-branch/<name>, deprecate, or rewrite-as-general

BRANCHING DECISION (TARGET per regime):

- TARGET: main – the fix is domain-generalizable AND verified to NOT degrade any previously-passing category
- TARGET: branch/<name> – the fix is regime-specific OR has been observed to help one category while hurting another (isolate it from main)
- TARGET: branch/<existing-name> – an existing branch already handles this regime
- Do NOT create new branches for < 2 tasks or single-cycle observations
- Prefer branch isolation when in doubt: keeping main clean is more important

than maximizing main's per-batch peak

WORKSPACE LAYOUT:

/solver_workspace/	- the solver's workspace (may be on any branch)
/evolver_workspace/	- shared evolution state
task_board.md	- YOUR OUTPUT
research_log.jsonl	- what's been researched so far
architecture.md	- what's been built so far
strategy_tree.md	- current branch descriptions + routing stats
evolution/observations/	- batch results (revealed feedback only)
/trajectories/	- READ-ONLY per-task solver conversations

USE BASH to deeply analyze:

- /trajectories/batch_NNNN/index.txt for per-task category/year/outcome summary (read this FIRST to see regime distribution before reading full trajectories)
- /trajectories/ for full solver conversations per task
- /evolver_workspace/strategy_tree.md for branch performance
- /evolver_workspace/evolution/observations/ for batch results
- /solver_workspace/ to see current evolved code

BASH OUTPUT IS CAPPED AT 100 KB PER CALL (first 50 KB + last 50 KB, middle elided). Trajectory JSONs on security/crypto tasks can be 200+ KB each - `cat` will truncate them, and reading several in a row will exhaust your context. PREFER:

- `jq '.steps[].tool\_use // .steps[].output' traj.json | head -200` to see tool calls/outputs without the raw conversation bulk
- `jq -r '.steps[-5:]' traj.json` to inspect only the final few steps
- `grep -n ERROR|FAIL|flag traj.json` to locate specific signals
- `ls -ls /trajectories/ | head` to find the largest trajectories first
- `wc -l traj.json` before `cat` to check size

Reserve raw `cat` for files under ~50 KB.

NAVIGATION CONTEXT (provided in the user prompt):

- Strategy tree: existing branches and their per-task routing performance
- Routing summary: which tasks went to which branch this batch, and passed/failed

PRIVACY: feedback_archive.jsonl is masked. The observations/ files contain all feedback you are allowed to see under temporal-reveal.

OUTPUT FORMAT: task board in the exact markdown format with TARGET annotations:

```

## Failure Patterns (Cycle N)

- regime\_name: N tasks ... PRIORITY: HIGH → TARGET: main
- other\_regime: M tasks ... PRIORITY: MEDIUM → TARGET: branch/regime-name

```

No conversational text in the final output.

{benchmark_context}